

CSCI 49900: Capstone

Group 8: Alex Haskel, James Lacoste, Kailun You



Overview

- Everyone uses LLMs
- However, it's difficult to understand how they work
- Our project is an interactive platform to demonstrate the mechanics of large language model inference optimization and to allow for the exploration of model behavior through internal interventions. The website will allow: Prompt → fixed baseline + configurable experimental run → side-by-side outputs + metrics.
- The project requires writing code to optimize a model, and creating an interactive platform in which to test model behavior and performance.
- Alex: Backend + interventions, James: Backend + inference optimization, Kailun: Frontend.



Model Selection (Alex)



Qwen3



Pythia



Model Family	Total Parameters	Observations
 Pythia	140M	Very light and fast/Responses aren't very interesting
 Pythia	410M	Responses somewhat more elaborate than 140M model.
 Pythia	1B	Best responses out of Pythia (still next token only)
 Qwen3 Base	600M	Interesting responses, still next token only.
 Qwen3 Post-trained	600M	Best for now imo, tuned for chatting, more interesting. Better training data: 36T tokens.
TinyLlama-v1.1	1.1B	Poor responses, meant for fine tuning
TinyLlama-Chat-v1.0	1.1B	Ok responses, chat tuned version of TinyLlama. Trained on far less data than Qwen3: 3T tokens.

Experiments (Alex)

Decode parameters

- Temperature
- TopK
- TopP
- Repetition penalty

Inference parameters

- Ablate MLP units
- Ablate attention heads
- Possibly modifying weights in other ways



MLP unit ablation example:

```
messages = [  
    {"role": "user", "content": "The capital of France is"},  
]
```

Okay, the user is asking about the capital of France. Let me recall... I know the capital is Paris. But wait, maybe I should confirm to make sure. Sometimes people confuse it with other countries.

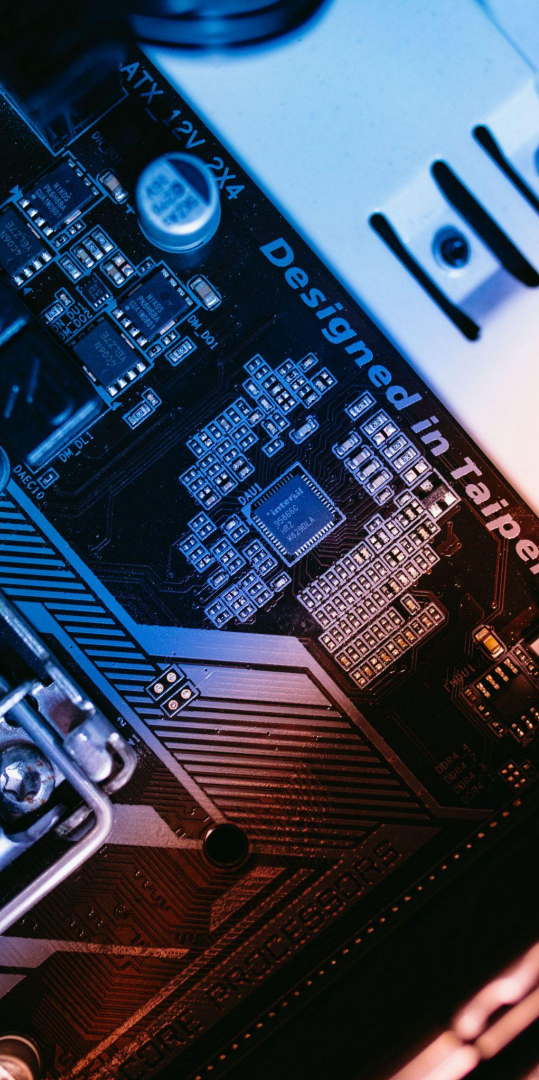
Zeroed activations in 100.0% of units in layer 0

Okay, the user provided a query in Arabic: "The capital of France is ..." and then said it's supposed to be translated as "the capital of China is ...". Now I need to figure out how to respond.



API contract (Alex)

- Need to define which information the frontend sends to the backend, and what is returned.
- Basic flow: User inputs prompts and parameters » Frontend POST request w/JSON body » Backend receives request and runs baseline and experimental condition inference on GPU » Backend responds to request » Frontend displays outputs and metrics.
- The front end should be able to take in a user's prompt, possibly a system prompt, and a list of parameters. The backend should generate a response using a baseline model, and a response using the user's chosen parameters, plus baseline and post-intervention metrics, so these can be displayed.



Cost Analysis (Alex)

- Model parameters are usually 16-bit floating point numbers (FP16). $16\text{bits} = 2\text{ bytes}$.
- GPU memory needed approx:
 $\text{Parameters} * 2\text{ bytes} + \text{overhead}$
- E.g., for 1B parameters, we need a GPU with at least 2 billion bytes = 2GB VRAM
- Cheapest GPUs (16GB) should suffice. On Runpod, a 16GB GPU costs \$0.58/hr, which seems OK. Need to put some limits on it just in case. Possibly could use bigger models
- Later, will need to look into concurrency and test.

Kailun You

Responsibility: Frontend

Currently...

01

Wireframe

Website layout design and planning.

02

Coding Language

Understanding what language needed for this project so it won't conflict with each other.
(REACT with JS that uses [D3.JS](#))
(Maybe Zustand for storing Info)

03

Learning

Learning [D3.JS](#) and its applications and relearn React and Zustand.



Kailun You

Responsibility: Frontend

Planning...

01

Fully Develop Wireframe

02

Improving in D3.js

Kailun You

Responsibility: Frontend

Problem...

01

Time needed to learn D3.JS

I currently do not know how much or how
quick I can learn D3.js

Prompt

Explain why the sky is blue in two sentences.

Run comparison

Shared by both runs

temperature 0.7

top-p 0.8

top-k 20

seed 42

max tokens 512

thinking off

Baseline

16-bit · no ablation

The sky looks blue because sunlight scatters off tiny gas molecules in the air. Shorter blue wavelengths scatter far more than red ones, so blue light reaches your eyes from every direction.

Your model

INT8 · 4 heads off · zero ablation

The sky looks blue because sunlight scatters off tiny gas molecules in the air. Shorter blue light is absorbed by the ozone layer, which reflects it back down across the whole sky.

Shading: how differently your model predicts each token low mid high first token that differs

Hover any token to inspect it

Selected token

which

Your model · position 24 · token id 29398

Baseline probability

2.0%

Your model probability

34.0%

Divergence at this position (KL)

0.85

Top candidates in your model

which

34.0%

the

27.7%

light

15.8%

a

9.2%

blue

5.3%

Decode speed

— tok/s

Ceiling 161 tok/s

Time to first token

— ms

Measured live

Weight memory

1.19 GB

Incl. shared embedding

Decode speed

— tok/s

Ceiling 255 tok/s

Time to first token

— ms

Measured live

Weight memory

0.75 GB

Incl. shared embedding

Your model

Weight precision

16-bit

INT8

INT4

Ablation mode

Zero

Mean

Turn off

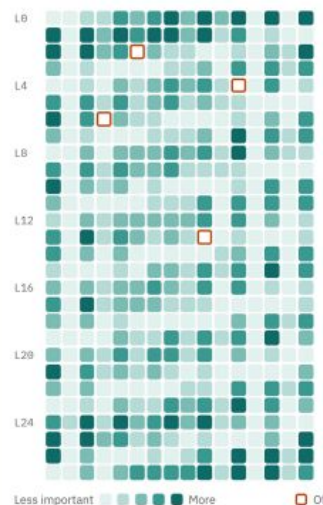
Query heads

KV groups

Attention heads

Reset

28 layers × 16 query heads, shaded by importance at INT8 from the head-importance experiment. Click a head to turn it off.



How quantization works

One row of a weight matrix, stored at lower precision. Each weight becomes a small integer times one scale shared by the whole row. Pick a bit width to see what the kernel reads from memory and what it costs in accuracy.

Bit width

8-bit (INT8)

4-bit (INT4)

2-bit

$$\text{scale} = \max|w| \div 127 = 0.00169 \quad q = \text{round}(w \div \text{scale}) \quad w \approx q \times \text{scale}$$

Original weight

16-bit, as stored today

0.0310	-0.0470	0.0120	0.0680	-0.0090	-0.0830	0.0540	0.0020	-0.0260	0.0710	-0.0610	0.2140	0.0190	-0.0380	0.0450	-0.0150
--------	---------	--------	--------	---------	---------	--------	--------	---------	--------	---------	--------	--------	---------	--------	---------

Stored integer

8-bit, what the kernel reads

18	-28	7	40	-5	-49	32	1	-15	42	-36	127	11	-23	27	-9
----	-----	---	----	----	-----	----	---	-----	----	-----	-----	----	-----	----	----

Reconstructed

integer $\times$ scale, in registers

0.0303	-0.0472	0.0118	0.0674	-0.0084	-0.0826	0.0539	0.0017	-0.0253	0.0708	-0.0607	0.2140	0.0185	-0.0388	0.0455	-0.0152
--------	---------	--------	--------	---------	---------	--------	--------	---------	--------	---------	--------	--------	---------	--------	---------

Error

reconstructed - original

-0.0007	-0.0002	-0.0002	-0.0006	+0.0006	+0.0004	-0.0001	-0.0003	+0.0007	-0.0002	+0.0003	+0.0000	-0.0005	-0.0008	+0.0005	-0.0002
---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------	---------

Storage per weight

1 byte

16-bit uses 2 bytes

Integer range

-127 to 127

Symmetric, no zero-point

Levels this row uses

16 of 255

Out of the range above

Mean error

0.0004

Average absolute error

Largest error

0.0008

At most half a step

Linear-layer weights

0.44 GB

0.88 GB at 16-bit

At 8 bits, the outlier barely matters

The largest weight (0.214, highlighted) sets the step size for the whole row, but 255 levels are enough that every other weight still lands within 0.0008 of its original value. This is why weight-only INT8 costs almost nothing in quality.

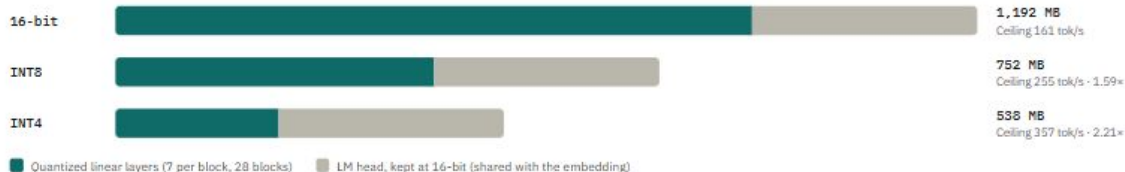
Why INT8 decodes faster

Generating one token reads every weight in the model once and does about one multiply-add per byte read. The GPU spends its time waiting on memory, not doing math, so fewer bytes per weight means more tokens per second.

Predicted, not measured: calculated from the model's layer sizes and the GPU's rated 192 GB/s. Measured values from the benchmark harness will sit beside these.

Bytes read per generated token

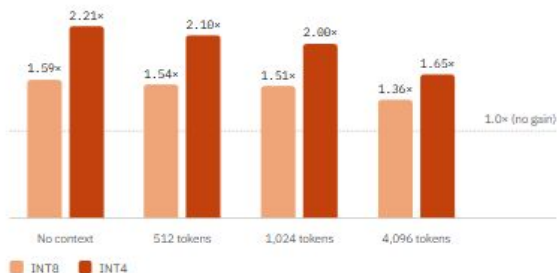
Qwen3-0.6B, weights only. Ceilings assume the RTX 4050 Laptop's 192 GB/s.



The LM head stays at 16-bit, so INT8 cuts the bytes per token by 37%, not 50%. INT4 cuts them by 55%.

The ceiling shrinks as context grows

Maximum decode speedup over 16-bit, from bytes read per token



Every step also reads the KV cache, 115 KB per token of context, and that cost is the same at every weight precision. At 4,096 tokens it adds 63% on top of INT8's weight traffic.

Kernel time per layer shape, INT8

Ideal time is weight bytes ÷ 192 GB/s. Measured times come from the benchmark harness.

Layer	N × K	Bytes	Ideal	Measured	
q_proj	2048 × 1024	2.10 MB	10.9 μs		[task 2.K7]
k_proj, v_proj	1024 × 1024	1.05 MB	5.5 μs		[task 2.K7]
o_proj	1024 × 2048	2.10 MB	10.9 μs		[task 2.K7]
gate_proj, up_proj	3072 × 1024	3.15 MB	16.4 μs		[task 2.K7]
down_proj	1024 × 3072	3.15 MB	16.4 μs		[task 2.K7]

Ideal ÷ measured is the kernel's share of peak bandwidth. Reading one 3 MB matrix takes about 16 μs on this GPU. On a 1 TB/s card it would take about 3 μs, close to the cost of launching the kernel at all.